

Encoding Categorical Variables: A Deep Dive into Target Encoding

Data comes in different shapes and forms. One of those shapes and forms is known as categorical data.

Juan Jose Munoz

Feb 5, 2024

11 min read

This poses a problem because most Machine Learning algorithms use only numerical data as input. However, categorical data is usually not a challenge to deal with, thanks to simple, well-defined functions that transform them into numerical values. If you have taken any data science course, you will be familiar with the one hot encoding strategy for categorical features. This strategy is great when your features have limited categories. However, you will run into some issues when dealing with high cardinal features (features with many categories). Here is how you can use target encoding to transform Categorical features into numerical values.

The problem with One Hot encoding

Early in any data science course, you are introduced to one hot encoding as a key strategy to deal with categorical values, and rightfully so, as this strategy works really well on low cardinal features (features with limited categories).

In a nutshell, One hot encoding transforms each category into a binary vector, where the corresponding category is marked as 'True' or '1', and all other categories are marked with 'False' or '0'.

```
import pandas as pd
```

```
# Sample categorical data
```

```
data = {'Category': ['Red', 'Green', 'Blue', 'Red', 'Green']}
```

```
# Create a DataFrame
```

```
df = pd.DataFrame(data)
```

```
# Perform one-hot encoding
```

```
one_hot_encoded = pd.get_dummies(df['Category'])
```

```
# Display the result
```

```
print(one_hot_encoded)
```

	Blue	Green	Red
0	False	False	True
1	False	True	False
2	True	False	False
3	False	False	True
4	False	True	False

One hot encoding output – we could improve this by dropping one column because if we know Blue and Green, we can figure the value of Red. Image by author

While this works great for features with limited categories (Less than 10–20 categories), as the number of categories increases, the one-hot encoded vectors become longer and sparser, potentially leading to increased memory usage and computational complexity, let's look at an example.

The below code uses Amazon Employee Access data, made publicly available in kaggle: <https://www.kaggle.com/datasets/lucamassaron/amazon-employee-access-challenge>

The data contains eight categorical feature columns indicating characteristics of the required resource, role, and workgroup of the employee at Amazon.

data.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32769 entries, 0 to 32768
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ACTION                32769 non-null  object
1   RESOURCE              32769 non-null  object
2   MGR_ID                32769 non-null  object
3   ROLE_ROLLUP_1         32769 non-null  object
4   ROLE_ROLLUP_2         32769 non-null  object
5   ROLE_DEPTNAME         32769 non-null  object
6   ROLE_TITLE            32769 non-null  object
7   ROLE_FAMILY_DESC      32769 non-null  object
8   ROLE_FAMILY           32769 non-null  object
9   ROLE_CODE             32769 non-null  object
```

Column information. Image by author

Display the number of unique values in each column

unique_values_per_column = data.nunique()

print("Number of unique values in each column:")

print(unique_values_per_column)

Number of unique values in each column:

```
ACTION                2
RESOURCE              7518
MGR_ID               4243
ROLE_ROLLUP_1         128
ROLE_ROLLUP_2         177
ROLE_DEPTNAME         449
ROLE_TITLE            343
ROLE_FAMILY_DESC     2358
ROLE_FAMILY           67
ROLE_CODE             343
dtype: int64
```

The eight features have high cardinality. Image by author

Using one hot encoding could be challenging in a dataset like this due to the high number of distinct categories for each feature.

#Initial data memory usage

```
memory_usage = data.memory_usage(deep=True)
```

```
total_memory_usage = memory_usage.sum()
```

```
print(f"Total memory usage of the DataFrame: {total_memory_usage / (1024 ** 2):.2f} MB")
```

Total memory usage of the DataFrame: 11.24 MB

The initial dataset is 11.24 MB. Image by author

#one-hot encoding categorical features

```
data_encoded = pd.get_dummies(data,
                               columns=data.select_dtypes(include='object').columns,
                               drop_first=True)
```

```
data_encoded.shape
```

```
(32769, 15618)
```

After one-hot encoding, the dataset has 15 618 columns. Image by author

	ACTION_1	RESOURCE_38	RESOURCE_136	RESOURCE_138	RESOURCE_153	RESOURCE_199	RESOURCE_203	RESOURCE_216	RESOURCE_233	RESOURCE_237	...	ROLE_CODE_212194	ROLE_CODE_216827	ROLE_CODE_239004	ROLE_CODE_240105	ROLE_CODE_247660	ROLE_CODE_254396
0	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
3	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
4	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0

5 rows × 15618 columns

The resulting data set is highly sparse, meaning it contains a lot of 0s and 1. Image by author

Memory usage for the one-hot encoded dataset

```
memory_usage = data_encoded.memory_usage(deep=True)
```

```
total_memory_usage = memory_usage.sum()
```

```
print(f"Total memory usage of the DataFrame: {total_memory_usage / (1024 ** 2):.2f} MB")
```

Total memory usage of the DataFrame: 488.08 MB

Dataset memory usage increased to 488.08 MB due to the increased number of columns. Image by author

As you can see, one-hot encoding is not a viable solution to deal with high cardinal categorical features, as it significantly increases the size of the dataset.

In cases with high cardinal features, target encoding is a better option.

Target encoding – overview of basic principle

Target encoding transforms a categorical feature into a numeric feature without adding any extra columns, avoiding turning the dataset into a larger and sparser dataset.

Target encoding works by converting each category of a categorical feature into its corresponding expected value. The approach to calculating the expected value will depend on the value you are trying to predict.

For Regression problems, the expected value is simply the average value for that category.

For Classification problems, the expected value is the conditional probability given that category.

In both cases, we can get the results by simply using the 'group_by' function in pandas.

#Example of how to calculate the expected value for Target encoding of a Binary outcome

```
expected_values = data.groupby('ROLE_TITLE')  
['ACTION'].value_counts(normalize=True).unstack()  
expected_values
```

ACTION	0	1
ROLE_TITLE		
117879	0.110669	0.889331
117885	0.032258	0.967742
117896	0.030303	0.969697
117899	0.075000	0.925000
117905	0.032375	0.967625
...	...	...
297560	NaN	1.000000
299559	0.090909	0.909091
307024	0.077088	0.922912
310825	NaN	1.000000
311867	NaN	1.000000

343 rows × 2 columns

The resulting table indicates the probability of each ACTION outcome by unique Role_title ID.

Image by author

The resulting table indicates the probability of each "ACTION" outcome by unique "_ROLETITLE" id. All that is left to do is replace the "_ROLETITLE" id with the values from the probability of "ACTION" being 1 in the original dataset. (i.e instead of category 117879 the dataset will show 0.889331)

While this can give us an intuition of how target encoding works, using this simple method runs the risk of overfitting. Especially for rare categories, as in those cases, target encoding will essentially provide the target value to the model. Also, the above method can only deal with seen categories, so if your test data has a new category, it won't be able to handle it.

To avoid those errors, you need to make the target encoding transformer more robust.

Defining a Target encoding class

To make target encoding more robust, you can create a custom transformer class and integrate it with scikit-learn so that it can be used in any model pipeline.

NOTE: The below code is taken from the book "The Kaggle Book" and can be found in Kaggle: <https://www.kaggle.com/code/lucamassaron/meta-features-and-target-encoding>

```
import numpy as np
import pandas as pd
```

```
from sklearn.base import BaseEstimator, TransformerMixin
```

```
class TargetEncode(BaseEstimator, TransformerMixin):
```

```
    def __init__(self, categories='auto', k=1, f=1,
                 noise_level=0, random_state=None):
        if type(categories)==str and categories!='auto':
            self.categories = [categories]
        else:
            self.categories = categories
        self.k = k
        self.f = f
        self.noise_level = noise_level
        self.encodings = dict()
        self.prior = None
        self.random_state = random_state

    def add_noise(self, series, noise_level):
        return series * (1 + noise_level *
                        np.random.randn(len(series)))

    def fit(self, X, y=None):
        if type(self.categories)=='auto':
            self.categories = np.where(X.dtypes == type(object()))[0]

        temp = X.loc[:, self.categories].copy()
        temp['target'] = y
        self.prior = np.mean(y)
        for variable in self.categories:
            avg = (temp.groupby(by=variable)['target']
                  .agg(['mean', 'count']))
            # Compute smoothing
            smoothing = (1 / (1 + np.exp(-(avg['count'] - self.k) /
                                           self.f)))
            # The bigger the count the less full_avg is accounted
            self.encodings[variable] = dict(self.prior * (1 -
                                                           smoothing) + avg['mean'] * smoothing)

        return self

    def transform(self, X):
        Xt = X.copy()
        for variable in self.categories:
            Xt[variable].replace(self.encodings[variable],
                               inplace=True)
            unknown_value = {value:self.prior for value in
                             X[variable].unique()
                             if value not in
                             self.encodings[variable].keys()}
            if len(unknown_value) > 0:
                Xt[variable].replace(unknown_value, inplace=True)
        Xt[variable] = Xt[variable].astype(float)
        if self.noise_level > 0:
            if self.random_state is not None:
                np.random.seed(self.random_state)
```

```

        Xt[variable] = self.add_noise(Xt[variable],
                                      self.noise_level)
    return Xt

```

```

def fit_transform(self, X, y=None):
    self.fit(X, y)
    return self.transform(X)

```

It might look daunting at first, but let's break down each part of the code to understand how to create a robust Target encoder.

Class Definition

```
class TargetEncode(BaseEstimator, TransformerMixin):
```

This first step ensures that you can use this transformer class in scikit-learn pipelines for data preprocessing, feature engineering, and machine learning workflows. It achieves this by inheriting the scikit-learn classes `BaseEstimator` and `TransformerMixin`.

Inheritance allows the `TargetEncode` class to reuse or override methods and attributes defined in the base classes, in this case, `BaseEstimator` and `TransformerMixin`.

`BaseEstimator` is a base class for all scikit-learn estimators. Estimators are objects in scikit-learn with a "fit" method for training on data and a "predict" method for making predictions.

`TransformerMixin` is a mixin class for transformers in scikit-learn, it provides additional methods such as "fit_transform", which combines fitting and transforming in a single step.

Inheriting from `BaseEstimator` & `TransformerMixin`, allows `TargetEncode` to implement these methods, making it compatible with the scikit-learn API.

Defining the constructor

```

def __init__(self, categories='auto', k=1, f=1,
             noise_level=0, random_state=None):
    if type(categories)==str and categories!='auto':
        self.categories = [categories]
    else:
        self.categories = categories
    self.k = k
    self.f = f
    self.noise_level = noise_level
    self.encodings = dict()
    self.prior = None
    self.random_state = random_state

```

This second step defines the constructor for the "TargetEncode" class and initializes the instance variables with default or user-specified values.

The "categories" parameter determines which columns in the input data should be considered as categorical variables for target encoding. It is Set by default to 'auto' to automatically identify categorical columns during the fitting process.

The parameters k, f, and noise_level control the smoothing effect during target encoding and the level of noise added during transformation.

Adding noise

This next step is very important to avoid overfitting.

```

def add_noise(self, series, noise_level):
    return series * (1 + noise_level *
                    np.random.randn(len(series)))

```

The "_addnoise" method adds random noise to introduce variability and prevent overfitting during the transformation phase.

"np.random.randn(len(series))" generates an array of random numbers from a standard normal distribution (mean = 0, standard deviation = 1).

Multiplying this array by "_noise_level" scales the random noise based on the specified noise level."

This step contributes to the robustness and generalization capabilities of the target encoding process.

Fitting the Target encoder

This part of the code trains the target encoder on the provided data by calculating the target encodings for categorical columns and storing them for later use during transformation.

```

def fit(self, X, y=None):
    if type(self.categories)=='auto':

```

```

self.categories = np.where(X.dtypes == type(object()))[0]

temp = X.loc[:, self.categories].copy()
temp['target'] = y
self.prior = np.mean(y)
for variable in self.categories:
    avg = (temp.groupby(by=variable)['target']
           .agg(['mean', 'count']))
    # Compute smoothing
    smoothing = (1 / (1 + np.exp(-(avg['count'] - self.k) /
                                   self.f)))
    # The bigger the count the less full_avg is accounted
    self.encodings[variable] = dict(self.prior * (1 -
                                                  smoothing) + avg['mean'] * smoothing)

```

The smoothing term helps prevent overfitting, especially when dealing with categories with small samples.

The method follows the scikit-learn convention for fit methods in transformers.

It starts by checking and identifying the categorical columns and creating a temporary DataFrame, containing only the selected categorical columns from the input X and the target variable y.

The prior mean of the target variable is calculated and stored in the prior attribute. This represents the overall mean of the target variable across the entire dataset.

Then, it calculates the mean and count of the target variable for each category using the group-by method, as seen previously.

There is an additional smoothing step to prevent overfitting on categories with small numbers of samples. Smoothing is calculated based on the number of samples in each category. The larger the count, the less the smoothing effect.

The calculated encodings for each category in the current variable are stored in the encodings dictionary. This dictionary will be used later during the transformation phase.

Transforming the data

This part of the code replaces the original categorical values with their corresponding target-encoded values stored in self.encodings.

```

def transform(self, X):
    Xt = X.copy()
    for variable in self.categories:
        Xt[variable].replace(self.encodings[variable],
                             inplace=True)
        unknown_value = {value:self.prior for value in
                          X[variable].unique()
                          if value not in
                          self.encodings[variable].keys()}
        if len(unknown_value) > 0:
            Xt[variable].replace(unknown_value, inplace=True)
        Xt[variable] = Xt[variable].astype(float)
        if self.noise_level > 0:
            if self.random_state is not None:
                np.random.seed(self.random_state)
            Xt[variable] = self.add_noise(Xt[variable],
                                           self.noise_level)

    return Xt

```

This step has an additional robustness check to ensure the target encoder can handle new or unseen categories. For those new or unknown categories, it replaces them with the mean of the target variable stored in the prior_mean variable.

If you need more robustness against overfitting, you can set up a _noiselevel greater than 0 to add random noise to the encoded values.

The _fittransform method combines the functionality of fitting and transforming the data by first fitting the transformer to the training data and then transforming it based on the calculated encodings.

Now that you understand how the code works, let's see it in action.

#Instantiate TargetEncode class


```
te = TargetEncode(categories='ROLE_TITLE')
te.fit(data, data['ACTION'])
te.transform(data[['ROLE_TITLE']])
```

ROLE_TITLE	
0	0.967625
1	0.962963
2	0.889331
3	0.920413
4	0.866667
...	...
32764	0.920413
32765	1.000000
32766	0.993099
32767	0.913706
32768	0.925216

Output with Target encoded Role title. Image by author

The Target encoder replaced each "_ROLETITLE" id with the probability of each category. Now, let's do the same for all features and check the memory usage after using Target Encoding.

```
y = data['ACTION']
features = data.drop('ACTION',axis=1)
```

```
te = TargetEncode(categories=features.columns)
te.fit(features,y)
te_data = te.transform(features)
```

```
te_data.head()
```

	RESOURCE	MGR_ID	ROLE_ROLLUP_1	ROLE_ROLLUP_2	ROLE_DEPTNAME	ROLE_TITLE	ROLE_FAMILY_DESC	ROLE_FAMILY	ROLE_CODE
0	0.993099	1.000000	0.949222	0.956148	0.958333	0.967625	0.933440	0.942350	0.967625
1	0.966667	0.999993	0.949222	0.969075	0.893082	0.962963	0.999999	0.947941	0.962963
2	0.984431	0.993099	0.918478	0.918478	0.923077	0.889331	0.939394	0.907815	0.889331
3	0.971055	1.000000	0.949222	0.969075	0.989474	0.920413	0.955788	0.942350	0.920413
4	0.999947	0.999981	0.931159	0.876812	0.755556	0.866667	0.947368	0.837017	0.866667

Output, Target encoded features. Image by author

```
memory_usage = te_data.memory_usage(deep=True)
total_memory_usage = memory_usage.sum()
print(f"Total memory usage of the DataFrame: {total_memory_usage / (1024 ** 2):.2f} MB")
Total memory usage of the DataFrame: 2.25 MB
```

The resulting dataset only uses 2.25 MB, compared to 488.08 MB from the one-hot encoder.
Image by author

Target encoding successfully transformed the categorical data into numerical without creating extra columns or increasing memory usage.

Target encoding with SciKitLearn API

So far we have created our own target encoder class, however you don't have to do this anymore. In scikit-learn version 1.3 release, somewhere around June 2023, they introduced the Target Encoder class to their API. Here is how you can use target encoding with Scikit Learn

```
from sklearn.preprocessing import TargetEncoder
```

```
#Splitting the data
y = data['ACTION']
features = data.drop('ACTION',axis=1)
```

```
#Specify the target type
te = TargetEncoder(smooth="auto",target_type='binary')
X_trans = te.fit_transform(features, y)
```

```
#Creating a Dataframe
```

```
features_encoded = pd.DataFrame(X_trans, columns = features.columns)
```

	RESOURCE	MGR_ID	ROLE_ROLLUP_1	ROLE_ROLLUP_2	ROLE_DEPTNAME	ROLE_TITLE	ROLE_FAMILY_DESC	ROLE_FAMILY	ROLE_CODE
0	1.000000	1.000000	0.949821	0.956531	0.945398	0.968919	0.935175	0.943523	0.968919
1	0.954112	1.000000	0.949268	0.969864	0.891334	0.984435	1.000000	0.950765	0.984435
2	1.000000	1.000000	0.906596	0.906596	0.926238	0.893511	0.929134	0.910753	0.893511
3	0.942094	1.000000	0.949821	0.969196	0.986035	0.921666	0.954722	0.943523	0.921666
4	1.000000	1.000000	0.942222	0.895994	0.800990	0.909977	1.000000	0.841305	0.909977
...	...	...	...	...	...	...	...	...	...
32764	0.886174	0.947128	0.949268	0.956396	0.986772	0.920191	0.955545	0.942678	0.920191
32765	1.000000	1.000000	0.958671	0.958671	1.000000	1.000000	1.000000	0.972140	1.000000
32766	0.954125	1.000000	0.949268	0.952176	1.000000	1.000000	0.942132	0.966427	1.000000
32767	1.000000	0.767290	0.732569	0.716266	0.857495	0.911184	0.859434	0.858119	0.911184
32768	0.893006	1.000000	0.914698	0.913644	0.895248	0.926038	1.000000	0.904992	0.926038

Output from sklearn Target Encoder transformation. Image by author

Note that we are getting slightly different results from the manual Target encoder class because of the smooth parameter and randomness on the noise level.

As you see, sklearn makes it easy to run target encoding transformations. However, it is important to understand how the transformation works under the hood first to understand and explain the output.

While Target encoding is a powerful encoding method, it's important to consider the specific requirements and characteristics of your dataset and choose the encoding method that best suits your needs and the requirements of the machine learning algorithm you plan to use.

References

- [1] Banachewicz, K. & Massaron, L. (2022). The Kaggle Book: Data Analysis and Machine Learning for Competitive Data Science. Packt>
- [2] Massaron, L. (2022, January). Amazon Employee Access Challenge. Retrieved February 1, 2024, from <https://www.kaggle.com/datasets/lucamassaron/amazon-employee-access-challenge>
- [3] Massaron, L. Meta-features and target encoding. Retrieved February 1, 2024, from <https://www.kaggle.com/luca-massaron/meta-features-and-target-encoding>
- [4] Scikit-learn.sklearn.preprocessing.TargetEncoder. In scikit-learn: Machine learning in Python (Version 1.3). Retrieved February 1, 2024, from <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.TargetEncoder.html>